

Parallel Inference Strategies for Real-Time Neural Upscaling

Kevin-Brandon Joseph Corbett

Student

Jacksonville, United States of America

n00970482@unf.edu

Abstract—This paper surveys how we run neural upscales fast enough for games across CPUs, GPUs, and neural accelerators (NPUs) with data, model, and pipeline parallelism. It connects distributed ML frameworks with special tricks and smart schedulers that keep asynchronous work flowing on heterogeneous hardware. Case studies of adaptive super-resolution running on GPUs, NPUs, and hybrid devices, which in this context are “neural arrays” that the next-gen PlayStation could ship. The core result is that well-designed parallel inference beats serial pipelines on both latency and energy, which matters for consoles, PCs, and handhelds.

Index Terms—distributed machine learning, neural arrays, artificial intelligence, model parallelism, data parallelism, and system architecture

I. INTRODUCTION

Modern games are starting to rely on AI upscaling to deliver higher fidelity without overwhelming the hardware. Super-resolution networks such as EDSR [1] and SRGAN [2] are used to rebuild sharp frames from lower-resolution inputs, which could possibly allow consoles to have higher visual quality. The real constraint is time. Consistently hitting that target requires parallel inference across all available computer units. CPUs, GPUs, and neural accelerators (NPUs) must work together using data parallelism, model parallelism, and pipeline parallelism. Recent real-time super-resolution systems demonstrate that deep models can reconstruct high-resolution frames from low-resolution inputs while preserving perceptual detail under tight latency constraints [3]. Neural supersampling approaches extend this idea by integrating reconstruction directly into the rendering pipeline and using temporal feedback to improve image stability during motion [4]. The main challenge is latency. A renderer running at 60 FPS has only 16.7 ms per frame, and upscaling must run alongside animation, shading, physics, and effects. To meet this, inference must be distributed across all compute units, making parallelism essential. Decomposing the model allows multiple hardware units to process different parts simultaneously, reducing overall latency. Next-generation consoles are moving in this direction. The consoles could combine neural arrays for machine learning. It will use dedicated units for ray and path tracing and universal compression to reduce bandwidth pressure. Features such as PlayStation Super Resolution depend on efficient scheduling and operator fusion in this mix. This is similar to how NVIDIA DLSS and AMD FSR coordinate distributed compute paths to remain real-time. Similar heterogeneous

scheduling strategies already appear in embedded systems research, where coordinated CPU/GPU pipelines are used to maintain deterministic real-time performance under multi-model workloads [4]. This survey examines how techniques from high-performance computing and distributed machine learning can be adapted to neural upscaling on consumer hardware. Frameworks originally developed for embedded and edge devices, such as throughput-boosting multi-DNN schedulers [5] or fairness-aware heterogeneous execution strategies [6] offer insight into how future rendering pipelines can support neural upscalers at frame rate without exceeding power or thermal limits. I combine core ideas from high-performance computing with production machine learning tools to make neural upscalers meet specs on consumer hardware. The work focuses on three questions:

- 1) How can data, model, and pipeline parallelism be composed to meet strict latency targets for real-time inference?
- 2) Which scheduling strategies keep heterogeneous devices fully utilized without excessive synchronization costs?
- 3) How do we maintain temporal stability and perceptual quality while staying within bandwidth and power limits?

Motivation for this paper lies at the intersection of machine learning, real-time rendering, and parallel systems. Techniques once limited to distributed training in data centers can now run on consoles. My passion for video games informs this perspective: just as there are classic books, shows, and movies, there are also games that stand the test of time and offer more than simple entertainment. Modern games are getting far more demanding and increasingly photorealistic. As a result, traditional consoles are struggling to keep up. To stay competitive, manufacturers have to pack in much more powerful CPUs and GPUs, which drives up the price of each console. A console was always the alternative to a super-powerful but expensive PC. With new technology, it might be possible to build a super-powerful console at a lower cost while still delivering high quality, using AI such as distributed machine learning. I wrote this survey for the engineers and researchers who will actually build the PlayStation 6 era. I target game developers, engine programmers, graphics specialists, and ML practitioners who need concrete, shipping-ready answers on how to make real-time neural rendering, upscaling, and frame generation fast,

power-efficient, and perfectly stable on next-gen console hardware. The upcoming PlayStation 6 architecture is being built from the ground up to make this entire approach possible. Its custom hardware for machine learning and ray tracing shows a clear, strategic commitment to that vision. As Mark Cerny, Lead System Architect at Sony, explains, the long-term goal is a future where "more and more of what you see on screen will be touched or enhanced by machine learning."

II. BACKGROUND

A. Neural upscaling and real-time constraints

Real-time neural upscaling relies on models that can reconstruct high-resolution images from low-resolution inputs while preserving perceptual detail under strict latency constraints. Recent mobile and embedded systems research shows that carefully optimized super-resolution networks can achieve sub-33ms end-to-end latency on constrained GPUs and NPUs, demonstrating that real-time neural enhancement is feasible even on power-limited hardware [3]. Deep learning models such as ESRGAN [7] and other Super-Resolution Generative Adversarial Networks predict a high-resolution frame from a low-resolution input. They learn fine spatial details and, when trained on video, temporal consistency across frames. In real-time rendering, each inference must finish within a few milliseconds. That requirement forces the computation graph to decompose so multiple units can work in parallel. Neural supersampling methods developed for graphics pipelines further integrate spatial reconstruction with temporal feedback to reduce aliasing and improve stability during motion [8]. These examples show that we can achieve real-time neural upscaling today, as long as we design the models to minimize computation and tightly couple execution to the underlying hardware. In interactive rendering, latency becomes the dominant constraint. Each inference stage must be completed within a few milliseconds while sharing time with shading, animation, physics, and post-processing. Meeting these requirements demands decomposing the computation graph so that multiple hardware units can process different segments of the workload in parallel.

B. Heterogeneous architectures

Real-time inference uses a mix of CPUs, GPUs, and neural accelerators such as NPUs or tensor cores. GPUs excel at data-parallel workloads due to their wide SIMD execution units. NPUs specialize in low-precision dense matrix math. Achieving real-time performance requires minimizing data transfer overhead, maintaining high kernel occupancy, and coordinating these components to prevent any unit from becoming a bottleneck. Research on heterogeneous scheduling demonstrates how tightly coordinated pipelines can maintain deterministic timing even under multi-model workloads. For example, DART arranges CPUs and GPUs into pipelined execution nodes to guarantee predictable response times for real-time inference [4]. Other systems show that throughput can be significantly increased by intelligently distributing layers across heterogeneous cores. OmniBoost, for instance,

uses model-aware partitioning to increase throughput under multi-DNN workloads [5]. At the same time, complementary techniques focus on balancing execution so that no model starves when multiple networks share the same hardware [6]. We originally built these frameworks for embedded devices, robots, and phones. However, the same core principles apply directly to consoles: we must run upscalers under brutal thermal, power, and latency limits while squeezing every last drop of performance from all available compute units.

C. Forms of parallelism

- 1) Data parallelism
- 2) Model parallelism
- 3) Pipeline parallelism

The right mix depends on bandwidth, model size, cache behavior, and the frame latency target. These forms of parallelism let super-resolution models scale across heterogeneous hardware while meeting real-time budgets.

III. SURVEY METHODOLOGY

I ran a structured literature review across UNF Library's research database. I looked for journals, conference proceedings, and workshops as well as other peer-reviewed papers. I screened about 25 references.

The scope of the present survey was refined to focus exclusively on methods reporting latency, throughput, or energy metrics relevant to real-time, interactive, or frame-based workloads such as neural upscaling and neural supersampling. Inclusion Criteria:

- Methods enabling distributed or parallel inference across CPUs, GPUs, NPUs, or heterogeneous cores
- Techniques applicable to CNN or transformer-based super-resolution or neural rendering models
- Studies reporting quantitative performance metrics under constraints comparable to real-time or near-real-time rendering pipelines.

Exclusion Criteria

- Were not peer-reviewed
- Used traditional (non-neural) upscaling methods
- Did not report latency, throughput, or energy results

Some search terms were *distributed machine learning*, *neural arrays*, *artificial intelligence*, *model parallelism*, *data parallelism*, *neural upscaling*, *super-resolution*, *real-time inference*, and *system architecture*.

I chose the final papers because they were related to distributed machine learning. Also, I picked them if they involved heterogeneous hardware and provided techniques relevant to neural upscaling.

IV. LITERATURE REVIEW

Neural upscaling depends on high-throughput inference that still fits inside a strict frame budget. The literature discusses and combines data parallelism, model parallelism, and pipeline parallelism. Modern systems frequently combine all three parallelism. Them working together is key when deploying super-resolution or neural supersampling networks on heterogeneous hardware.

A. Data Parallelism

Data parallelism is the way to go for faster inference. Each device handles a different region of the frame or a different batch while using the same model weights. Seminal work by Goyal et al. demonstrated that large-scale CNN training can scale nearly linearly under data-parallel execution, provided that synchronization and communication overhead are carefully managed [9]. Although their focus is training rather than inference, the core idea carries over directly to neural upscaling. If we can split frames into tiles or blocks, multiple devices can process them independently as long as boundary synchronization is controlled. We originally developed these ideas for training, but they work even better for inference: we split each frame into tiles or blocks and let multiple devices rip through them independently. Building on this idea, Li et al. introduced PyTorch DistributedDataParallel (DDP), which significantly reduces communication overhead by overlapping gradient synchronization with computation and by grouping communication into buckets [10]. While DDP is primarily a training-time framework, its communication-optimization strategies inform how inference-time tiling systems must handle boundary exchanges and data movement to maintain performance.

In real-time video super-resolution, we slash latency on power-limited GPUs and NPUs by splitting the frame into tiles or blocks and running inference on them separately. Ignatov et al. demonstrated that mobile VSR systems can reach real-time performance by decomposing frames into blocks small enough to fit into local memory while avoiding excessive overlap regions that would negate computational savings [3]. Their results highlight two critical factors:

- Tile overlap management
- Locality of memory access

For workloads running multiple DNNs across different devices, there is a framework called FairBoost that can help. It helps balance both speed and fairness [6]. It uses a reinforcement-learning scheduler and layer-level profiling to dynamically assign work. It follows the motto "no model gets left behind" while maintaining high overall performance.

This principle extends directly to neural rendering and neural supersampling systems such as MNSS. The framework uses spatial subdivisions to distribute work across hardware accelerators while maintaining temporal stability [8]. Both Ignatov et al. and Yang et al. agree on the importance of spatial decomposition, but they target different quality metrics and system integrations, leaving open questions about how a single data-parallel strategy might unify both solutions.

When implemented effectively, data parallelism offers one of the most scalable approaches to real-time neural upscaling, particularly for architectures dominated by convolutional operations. It remains a core ingredient in modern neural supersampling pipelines, console rendering architectures, and mobile-accelerated video enhancement systems.

However, there is still a gap in systematically characterizing how data-parallel tiling strategies behave under the strict frame

budgets and heterogeneous hardware configurations typical of next-generation consoles.

B. Model parallelism

Model parallelism distributes different parts of a neural network across multiple devices, with each device responsible for computing a subset of the model's layers. Unlike data parallelism, model parallelism partitions the computational graph itself. This approach is essential when the model is too large to fit into the memory limits of a single accelerator or when different layers map more efficiently to different hardware types.

The most influential work in this area comes from large-scale transformer research. Shueybi et al. introduced Megatron-LM, which demonstrates that tensor-slicing and head-wise partitioning of transformer blocks can scale model capacity nearly linearly across multiple GPUs [11]. Their tensor-split approach allows individual attention heads, MLP units, and projection matrices to be distributed across devices, effectively multiplying the available compute and memory bandwidth.

Building on this, Rajbhandari et al. proposed ZeRO-Infinity. Which shards model parameters, gradients, and optimizer states across CPU and GPU memory pools. This dramatically reduces memory pressure and enables massive models to run on hardware with limited GPU memory [12]. Although primarily oriented toward training, memory-sharding techniques now enable large-scale super-resolution transformers to run efficiently at inference time on resource-constrained systems on chip (SoCs). Compared to Megatron, ZeRO-Infinity trades compute efficiency for greater memory flexibility, enabling it to handle much larger models. However, it introduces additional data movement between the CPU and the GPU. This highlights a disagreement in the literature. Megatron prioritizes compute throughput, while ZeRO-Infinity prioritizes scalability under memory constraints.

More recent research explores how model parallelism interacts with heterogeneous hardware. OmniBoost, for example, allocates layers to the most suitable hardware type based on computational cost and device capability, improving throughput under multi-model workloads [5]. Fairness-aware scheduling further supports this approach by ensuring that individual models maintain predictable execution times even when sharing heterogeneous hardware resources [6]. For neural upscaling, the same principles apply: convolution-heavy early layers may execute efficiently on GPUs, while dense reconstruction modules or attention blocks may run more effectively on NPUs.

For super-resolution transformers, these methods let larger models fit within console memory limits. However, model parallelism does come with some new bottlenecks. These are increased activation traffic between devices, tighter synchronization, and sensitivity to interconnect bandwidth. These constraints remain underexplored in the literature, especially for latency-critical applications such as real-time neural upscaling on console hardware.

C. Pipeline parallelism

Pipeline parallelism divides a neural network into sequential stages, with each stage assigned to a different device. Unlike data or model parallelism. Where they duplicate or shard computation across devices, pipeline methods form an assembly line execution path where multiple frames flow through different stages of the network simultaneously. This approach becomes especially useful when the model contains a natural hierarchical structure or when different layers map more efficiently to different accelerators.

The foundational work in this area is GPipe. Huang et al. introduced GPipe and demonstrated that partitioning large convolutional and transformer models into sequential stages with synchronous gradient updates enables efficient training. [13] GPipe significantly reduces memory pressure by keeping only a subset of layers active at a time. However, its fully synchronous execution introduces idle time when pipeline stages are imbalanced. [14] It gets refined in the DeepSpeed Pipeline to improve throughput and memory usage. Together, GPipe and PipeDream highlight a recurring tension in the literature: synchronous pipelines offer determinism, whereas asynchronous pipelines do not.

Distributed Deep Neural Networks (DDNN) provide one of the clearest examples of pipeline-style execution across heterogeneous hardware. In DDNN, lightweight early layers run directly on local devices, while deeper, more expensive layers execute on the edge or cloud [15]. We then combine vertical partitioning with modern hierarchical scheduling to eliminate bottlenecks. This strategy maximizes the utilization of GPU, CPU, and NPU resources. We then turn the worst-case frame time from chaos into something that can reliably set the clock, even when multiple neural networks are competing for compute in the same frame. Hierarchical partitioning strategies further optimize cross-device pipelines by minimizing dependency stalls and ensuring predictable latency [16].

When we use hierarchical or dependency-aware graph partitioning, we eliminate idle bubbles, keep every GPU and NPU core busy, and lock frame-to-frame latency into a rock-solid, predictable number—no matter how complex the neural pipeline gets [17].

- Stage 1: Low-resolution input handling and early grid-parallel convolutions (GPU)
- Stage 2: Deep transformations or attention operations (NPU)
- Stage 3: Final reconstruction and post-processing (GPU or NPU)

While one frame is busy in Stage 2, the next frame is already tearing through Stage 1, forming a streaming pipeline. The whole thing only stays fast if we keep every stage roughly the same length and move as little data as possible between the GPU and the NPU. Balance those two things, and the frames keep flowing.

Overall, the literature on pipeline parallelism agrees that sequential partitioning can significantly increase throughput and reduce memory pressure. However, it also reveals unresolved

challenges in latency predictability, cross-stage synchronization, and communication overhead. These gaps become especially important in real-time neural upscaling, where even small stalls can violate strict frame budgets on consoles.

D. Hybrid and heterogeneous approaches

State-of-the-art systems combine all three forms of parallelism and run them on mixed hardware. Additional strategies, such as operator-level optimizations, further accelerate inference in real-time settings. For example, Micikevicius et al. show that mixed-precision computation reduces memory bandwidth requirements and accelerates compute throughput, especially when paired with data and model parallelism [18] demonstrated that mixed-precision computation lowers memory bandwidth and accelerates computation. This technique is particularly effective when combined with data and model parallelism strategies.

Fine-grained micro-split frameworks show that breaking networks into sub-layer partitions can distribute computation across heterogeneous IoT hardware while minimizing latency [15]. Hierarchical partitioning automatically finds the sweet-spot split points inside a neural network. It uses the exact layers where handing off a tiny, compressed tensor to another accelerator costs far less in bandwidth and memory than forcing everything onto a single unit [16]. At runtime, the system monitors real-world conditions and dynamically decides which accelerator should run each stage, precisely as distributed edge-cloud schedulers already do today [19]. EdgeFlow is a strong example of this approach. It breaks DAG-structured models into smaller execution units. It manages the flow of intermediate results across edge devices, helping to lower latency and increase parallelism for complex model structures [19].

Operator-level optimizations like aggressive quantization, kernel fusion, and custom low-precision kernels. These tricks slash memory traffic and boost throughput while still allowing for high fidelity graphics [20]. When combined with tiled execution, model sharding, and pipelined inference, these optimizations make real-time super-resolution feasible on CPU, GPU, and NPU architectures.

Frameworks like DISNET [21] take this a step further, acting like a head chef or traffic cop that looks at every task, intelligently partitions work, and sends each piece to the appropriate specialized hardware. The result is that even the most demanding multi-DNN pipelines now execute efficiently and predictably. This orchestration enables even complex workloads, such as multi-DNN real-time rendering, to run efficiently on heterogeneous devices and enables game worlds to feature NPCs that act with believable intelligence.

Together, scalable neural upscaling depends on the convergence of parallel algorithms, intelligent scheduling frameworks, and hardware built for machine learning rendering. This combined trajectory defines the future of scalable neural upscaling on next-generation platforms.

V. CONTRIBUTIONS

The survey links data, models, and pipeline parallelism from ML research to practical pipelines used in real-time graphics. It unites operator-level methods such as quantization, tiling, and kernel fusion with higher-level forms of parallelism. The result is a concrete, reproducible playbook that finally lets developers ship photorealistic 4K/120 fps games with full ray tracing and deeply interactive, AI-driven worlds all on consumer hardware that stays cool, quiet, and affordable.

It is in tying low-level kernel tricks directly to high-level, frame-by-frame scheduling decisions across the GPU, tensor cores, and dedicated NPU. When those two worlds talk to each other properly micro-optimizations stop being isolated wins and instead become part of a system-wide strategy that delivers perfectly predictable, sub-millisecond frame times even when half a dozen neural networks are all screaming for cycles at once.

That tight coupling is exactly what Project Amethyst set out to solve, and it is what the console architect will build around the PlayStation 6. Instead of a patchwork of point solutions (DART for real-time guarantees, OmniBoost for clever layer placement, HiDP for hierarchical scheduling), each only owns one corner of the threefold challenge. The PS6 vision is to stop picking one and instead collapse all three solutions into a single, console-native orchestrator that works.

Synthesizing insights from high-performance computing, ML systems design, and neural graphics, this survey provides a forward-looking blueprint for engineers developing next-generation upscaling systems.

PS6 Feature	The Challenge	The Breakthrough Tech
Hyper-Realistic NPCs	Too many AIs slow the system.	FairBoost : Smartly assigns AI tasks to the best chip.
Living, Evolving Worlds	One console can't simulate it all.	DISNET/DDNN : Splits the world between the console & cloud.
8K Graphics in Real-Time	Rendering native 8K is too hard.	Mobile VSR : AI upscaling predicts pixels intelligently.
No Loading Screens	Game data is too massive.	EdgeFlow : Partitions complex assets for parallel loading.

Fig. 1. Mapping distributed inference frameworks to potential PlayStation 6 features.

Next-gen consoles like the PlayStation 6 will pack specialized neural accelerators, advanced compression engines, and multi-core heterogeneous compute fabrics. They will be using massive DAG-structured models like YOLOv5 or ResNet-152 running smoothly on "small" consumer silicon with the help of distributed inference tricks from FairBoost, DDNN/DISNET, Mobile VSR, and EdgeFlow. These show exactly how real-time neural upscaling, asset streaming, and rendering pipelines split workloads dynamically so we can have affordable consoles that can deliver 8K, believable NPCs, and giant procedural worlds without melting the box or

missing frames. It's only the beginning; new technologies are emerging, making this more possible.

VI. EMERGING TECHNOLOGIES

Importance-driven MoE offloading means the chip wakes up only the neural networks an NPC actually needs in the moment [22]. Everything else sits in DRAM, costing almost nothing. Then you've got non-autoregressive, large-batch, speculative decoding, so the dialogue model isn't crawling out a few tokens at a time. It's actually firing off the tokens it needs per cycle, which means more NPCs talking all at once [23]. Next, we have runtime mixed-precision mixed with heterogeneous scheduling. This lets you drop attention to 4-bit on the NPU, keep the GPU workload down, and stash activations on the CPU [24]. Inter-frame encoder caching reuses about 60 to 80 percent of the last frame's activations, turning 8K neural rendering into a possibility [25]. Lastly, out-of-order block-level prefill on mobile NPUs takes those large token prompts that used to slow down the SoC and now do more for less. It does this by slicing everything into tiny blocks and slamming them in whatever order memory prefers [26]. Advancing these technologies will be critical for enabling lifelike NPC behavior, real-time neural rendering, and large-scale simulation on future platforms such as the anticipated PlayStation 6 that Mark Cerny envisioned.

VII. CONCLUSION

After many years, the end of the brute-force era is near. Simply piling on more teraflops has delivered diminishing returns for years. The cost, power, and thermal curves have turned vertical, while generational leaps have plateaued. The next revolution in entertainment will not come from doing more. It will come from intelligent orchestration built on top of parallel computing.

This survey has shown that the full arsenal of distributed-inference techniques, data, tensor, and pipeline parallelism; vertical, horizontal, and hierarchical partitioning; operator-level optimizations; mixed-precision execution; and heterogeneity-aware runtimes already exists in research form. What has been missing is their systematic integration into a single, console-native framework that treats the entire SoC as a collaborative edge cluster collapsed onto one die.

All-in-one devices powered by specialized neural hardware will deliver the high-quality gameplay and media that users expect from next-generation consoles, such as the anticipated PlayStation 6. My experiences helped me find ways to improve my overall gaming experience while researching. I wanted better graphics for lower costs and less computational power, as well as wanting games to do more with less. They represent the architectural pivot that finally resolves the next-generation trilemma of photorealistic visuals, extreme performance, and deep interactivity on consumer hardware that remains affordable, cool, and quiet. The techniques surveyed here, from classic DDNN vertical splitting to the latest hierarchical schedulers, provide the concrete, reproducible blueprint for this

future. This is where the era of working harder is over because the era of working smarter has begun.

VIII. CITATIONS AND BIBLIOGRAPHIES

ACKNOWLEDGMENT

I want to thank Scott Piersall for trusting me with this project and for the freedom to dive into such an expansive and important topic. What started as a quest to understand how future consoles can deliver breathtaking graphics without breaking the bank or the power meter has revealed a much bigger picture. Making a better console is also applicable to resource-constrained domains like medical devices, satellite payloads, and autonomous systems. Doing more with less isn't just the future of gaming but also the future of edge intelligence everywhere.

REFERENCES

- [1] B. Lim, S. Son, H. Kim, S. Nah, and K. M. Lee, "Enhanced deep residual networks for single image super-resolution," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPR Workshops)*. Honolulu, HI: IEEE, July 2017, pp. 1–8.
- [2] C. Ledig, L. Theis, F. Huszár, J. Caballero, A. Cunningham, A. Acosta, A. Aitken, A. Tejani, T. Topping, Z. Wang, and W. Shi, "Photo-realistic single image super-resolution using a generative adversarial network," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Honolulu, HI: IEEE, July 2017, pp. 4681–4690.
- [3] A. Ignatov, R. Timofte, K. Kim, S. Y. Yoon, T. Lee, and K. Sohn, "Real-time video super-resolution on smartphones with deep learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. IEEE, June 2021.
- [4] Y. Xiang and H. Kim, "Pipelined data-parallel cpu/gpu scheduling for multi-DNN real-time inference (DART)," in *Proceedings of the IEEE Real-Time Systems Symposium (RTSS)*. IEEE, December 2019, pp. 392–405.
- [5] A. Karatzas, D. Iosifidis, and I. Anagnostopoulos, "Omniboost: Boosting throughput of heterogeneous embedded devices under multi-DNN workload," in *Proceedings of the ACM/IEEE Design Automation Conference (DAC)*. ACM/IEEE, July 2023.
- [6] A. Karatzas and I. Anagnostopoulos, "Balancing throughput and fair execution of multi-DNN workloads on heterogeneous embedded devices," *IEEE Embedded Computing Letters*, 2025.
- [7] X. Wang, K. Yu, S. Wu, J. Gu, Y. Liu, C. Dong, Y. Qiao, and C. C. Loy, "Esrgan: Enhanced super-resolution generative adversarial networks," in *Proceedings of the European Conference on Computer Vision Workshops (ECCV Workshops)*. Munich, Germany: Springer, September 2018, pp. 63–79.
- [8] S. Yang, Q. Zhou, Y. Li, and X. Liu, "Mnss: Neural supersampling framework for real-time rendering on mobile devices," *IEEE Transactions on Visualization and Computer Graphics*, 2024.
- [9] P. Goyal, P. Dollár, R. Girshick, P. Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, and K. He, "Accurate, large minibatch sgd: Training imagenet in 1 hour," 2017. [Online]. Available: <https://arxiv.org/abs/1706.02677>
- [10] S. Li, Y. Zhao, R. Varma, O. Parkhi, and M. Matthew, "Pytorch distributed: Experiences on accelerating data-parallel training," *Proceedings of the VLDB Endowment (PVLDB)*, vol. 13, no. 12, pp. 3005–3018, December 2020.
- [11] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," in *International Conference on Learning Representations (ICLR)*. Addis Ababa, Ethiopia: OpenReview.net, April 2020.
- [12] S. Rajbhandari, J. Rasley, O. Rauf, and Y. He, "Zero-infinity: Breaking the gpu memory wall for extreme-scale deep learning," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC '21)*. St. Louis, MO: ACM Press, November 2021, pp. 1–18.
- [13] Y. Huang, Y. Cheng, A. Bhatnagar, Q. Song, N. Lille, Y. Kang, V. A. Korthikanti, and J. Demmel, "Gpipe: Efficient training of giant neural networks using pipeline parallelism," in *Advances in Neural Information Processing Systems (NeurIPS)*. Vancouver, BC, Canada: Curran Associates, Inc., December 2019, pp. 103–112.
- [14] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. Derészegi, G. Ganger, P. Gibbons, and M. Zaharia, "Pipedream: Generalized pipeline parallelism for dnn training," in *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*. Huntsville, ON, Canada: ACM Press, October 2019, pp. 1–15.
- [15] S. Teerapittayanon, B. McDanel, and H. T. Kung, "Distributed deep neural networks over the cloud, the edge, and end devices," in *Proceedings of the IEEE International Conference on Distributed Computing Systems (ICDCS)*. IEEE, June 2017, pp. 328–339.
- [16] F. N. Peccia and O. Bringmann, "Embedded distributed inference of deep neural networks: A systematic review," *Journal of Systems Architecture*, vol. 155, p. 103040, 2024.
- [17] Z. Taufique, A. Rahman, and M. Shirazi, "Hidp: Hierarchical DNN partitioning for distributed inference on heterogeneous edge platforms," in *Proceedings of the IEEE International Conference on Edge Computing (EDGE)*. IEEE, July 2024.
- [18] P. Micikevicius, S. Narang, A. Maietti, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," in *International Conference on Learning Representations (ICLR)*, Vancouver, BC, Canada, April 2018.
- [19] C. Hu and B. Li, "Distributed inference with deep learning models across heterogeneous edge devices," in *Proceedings of the IEEE International Conference on Computer Communications (INFOCOM)*. IEEE, May 2022.
- [20] L. Xiao, P. Guerrero, S. Z. Nowrouzezahrai, and D. M. Luebke, "Neural supersampling for real-time rendering," *ACM Transactions on Graphics*, vol. 39, no. 4, pp. 142:1–142:12, 2020.
- [21] E. Samikwa, T. Abdellatif, M. Abdelsalam, and N. Zhong, "Disnet: Distributed micro-split deep learning in heterogeneous dynamic iot," *IEEE Internet of Things Journal*, vol. 10, no. 12, pp. 10495–10507, 2023.
- [22] G. Zhu, M. Li, H. Dai, X. Liu, W. Wang, K. Li, J. xiao, L. Chen, and W. Wang, "Enabling moe on the edge via importance-driven expert scheduling," 2025. [Online]. Available: <https://arxiv.org/abs/2508.18983>
- [23] L. Shi, Z. Li, L. Zhang, B. Qi, G. Liu, and H. Zhao, "Scaling llm speculative decoding: Non-autoregressive forecasting in large-batch scenarios," 2025. [Online]. Available: <https://arxiv.org/abs/2511.20340>
- [24] L. Chen, D. Feng, E. Feng, Y. Wang, R. Zhao, Y. Xia, P. Xu, and H. Chen, "Characterizing mobile soc for accelerating heterogeneous llm inference," 2025. [Online]. Available: <https://arxiv.org/abs/2501.14794>
- [25] L. Liu and T. M. Aamodt, "Reframe: Layer caching for accelerated inference in real-time rendering," 2025. [Online]. Available: <https://arxiv.org/abs/2506.13814>
- [26] D. Xu, H. Zhang, L. Yang, R. Liu, G. Huang, M. Xu, and X. Liu, "Fast on-device llm inference with npus," 2024. [Online]. Available: <https://arxiv.org/abs/2407.05858>